

GUÍA TÉCNICA · FIWARE · NGSI-LD · SMART DATA MODELS

Smart Data Models explicados con un caso real

Del fichero GeoJSON de 104 parcelas de olivar a un modelo de datos estándar, validado y listo para cargar en un context broker: qué son los Smart Data Models, cómo se leen, cómo se diseñan y cómo se contribuye a ellos.

Caso de estudio: `parcelas_agronicas.geojson` (capa `_jmvisor`)

Modelo resultante: AgriParcel 1.1.0 (propuesta de extensión a `dataModel.Agrifood`) + AgriCrop + Device

Paquete asociado: `agriparcel-olivar-sdm.zip`

Fecha: septiembre de 2026



UNIVERSIDAD
DE
CÓRDOBA

Aula de Transformación Digital FIWARE · Universidad de Córdoba
Documento de formación interna

Índice

1	El punto de partida: datos sin modelo	por qué hace falta un modelo
2	Qué es un Smart Data Model	la iniciativa y su organización
3	NGSI-LD en lo imprescindible	entidades, atributos, @context
4	Anatomía de un modelo	los ficheros de un repositorio SDM
5	Método para diseñar un modelo	ocho pasos reutilizables
6	El caso paso a paso	del GeoJSON a las entidades
7	Las decisiones de diseño, explicadas	por qué cada elección
8	La misma parcela en cuatro formatos	key-values, normalizado, v2, LD
9	Validación y calidad de datos	el schema como contrato
10	El paquete generado y cómo usarlo	scripts, broker y consultas
11	Contribuir a Smart Data Models	del paquete al pull request
12	Glosario y lista de comprobación	referencia rápida

Cómo leer esta guía. Los capítulos 1 a 5 explican los conceptos de forma general y sirven para cualquier conjunto de datos. Los capítulos 6 a 11 aplican esos conceptos al fichero de parcelas de olivar y justifican cada decisión tomada. Si ya conoces NGSI-LD puedes saltar el capítulo 3; si solo quieres usar el paquete, ve directamente al 10.

1 El punto de partida: datos sin modelo

Un fichero de datos puede ser perfectamente correcto y, aun así, ser difícil de reutilizar por cualquiera que no sea su autor. Ese es el problema que resuelven los modelos de datos.

El fichero de partida es un **GeoJSON** con 104 parcelas de olivar. Cada parcela es una *Feature* con una geometría `MultiPolygon` en coordenadas WGS84 (longitud, latitud) y 21 propiedades. Así es una de ellas, tal como viene:

```
{ "id": 11, "nombre": "", "variedad": "HOJIBLANCA", "marco": "8x8", "fecha": "2012",  
  "cubiera": "si", "pol_arregl": 1, "area_ha": 9.7986, "area_m2": 97986.08,  
  "pendiente_media_pct": 5.3, "orientacion_grados": 287, "orientacion_card": "W",  
  "dist_siar_km": 9.74, "estacion_siar": "C0101 - IFAPA Centro de Cabra",  
  "n_recintos_catastro": 2, "rc_catastro": "14013A001001530000GH; 14013A001000510000GJ",  
  "sigpac_catastro": "14:013:0:0:1:153; 14:013:0:0:1:51",  
  "elev_media_m": 419.5, "elev_min_m": 405.0, "elev_max_m": 429.8,  
  "perfil_ns": [424.4, 422.9, 419.2, ... 14 valores] }
```

Propiedades originales de la parcela 11 (geometría omitida).

Para quien creó la capa, todo está claro. Pero imagina que otra persona, otro grupo de investigación o una aplicación tiene que usar estos datos sin preguntar nada. Se encontrará con problemas de varios tipos:

Nombres que solo entiende el autor

`pol_arregl`, `perfil_ns` o `cubiera` (con errata) no dicen por sí mismos qué contienen. Otra fuente llamaría a lo mismo `cover`, `cubierta_vegetal` o `cc`.

Redundancias

`area_ha` y `area_m2` dicen lo mismo; `orientacion_card` se deduce de los grados; `n_recintos_catastro` es la longitud de una lista. Si alguien corrige uno y no el otro, aparecen contradicciones.

Unidades implícitas

La unidad va escondida en el nombre (`_km`, `_pct`, `_m`). Un programa no puede leer eso de forma fiable.

Información repetida

El texto "C0101 - IFAPA Centro de Cabra" se repite 104 veces; la variedad, entre 1 y 50 veces. No hay un sitio único donde describir la estación o la variedad.

Valores compuestos en texto

"8x6" son dos números; "14:013:0:0:1:153; 14:013:0:0:1:51" es una lista. Para usarlos hay que trocear texto.

Sin contrato

Nada indica qué campos son obligatorios, qué valores son válidos o qué pasa si falta uno. Cada consumidor lo interpreta a su manera.

Un **modelo de datos** es un acuerdo explícito sobre todo esto: cómo se llama cada cosa, de qué tipo es, en qué unidad se expresa, qué es obligatorio y cómo se relacionan unas cosas con otras. Un **modelo de datos estándar** da un paso más: ese acuerdo no es solo tuyo, sino que lo comparte una comunidad, de manera que tus datos encajan con los de otros sin traducciones a medida.

Idea clave. El objetivo no es "cambiar de formato", sino hacer que el significado de los datos viaje con ellos. El GeoJSON ya es un formato estándar; lo que le falta es un vocabulario estándar.

2 Qué es un Smart Data Model

Smart Data Models es una iniciativa abierta, impulsada por la FIWARE Foundation junto con otras organizaciones, que publica modelos de datos comunes para ciudades, agricultura, energía, medio ambiente, industria y otros sectores, con licencia abierta y un proceso público de contribución.

Cada modelo describe un **tipo de entidad**: una parcela agrícola, un sensor, una observación meteorológica, un aparcamiento, un contenedor de residuos... Para cada tipo define sus atributos con nombre, tipo, unidad y descripción, y proporciona ejemplos listos para usar. Los modelos son independientes de la tecnología de almacenamiento, pero están pensados para encajar de forma natural con **NGSI-LD**, la API de gestión de información de contexto normalizada por ETSI que usan los context brokers de FIWARE.

2.1 Cómo está organizado

La organización es jerárquica y se refleja directamente en GitHub:

Dominio › **Subject** (un repositorio) › **Modelo** (un tipo de entidad) › **Atributos**

Nivel	Qué es	En nuestro caso
Dominio	Gran área temática	Smart AgriFood
Subject	Repositorio <code>dataModel.X</code> con modelos relacionados	<code>dataModel.Agrifood</code> (y <code>dataModel.Device</code>)
Modelo	Carpeta con el schema y los ejemplos de un tipo	<code>AgriParcel</code> , <code>AgriCrop</code> , <code>Device</code>
Atributo	Cada propiedad o relación del modelo	<code>area</code> , <code>hasAgriCrop</code> , <code>slope</code> ...

2.2 La familia Agrifood

El subject `dataModel.Agrifood` contiene modelos que se referencian entre sí para describir una explotación agraria completa. Los más relevantes para este caso son:

AgriFarm

La explotación: titular, dirección, conjunto de parcelas.

AgriSoil

Tipo y características del suelo, enlazado desde la parcela.

AgriParcel

Una parcela de terreno con su geometría, superficie, cultivo, suelo, dispositivos... Es el modelo central de nuestro caso.

AgriParcelRecord

Mediciones que cambian en el tiempo sobre una parcela (humedad de suelo, temperatura de hoja...).

AgriCrop

La descripción de un cultivo (o variedad): nombre, concepto AGROVOC, calendario, plagas, fertilizantes.

AgriParcelOperation

Operaciones realizadas: siembra, riego, tratamiento, cosecha.

Esta separación es importante: **AgriParcel describe lo que la parcela es** (casi estático), mientras que las mediciones y operaciones van en entidades aparte que la referencian. Nuestros datos son descriptivos, así que encajan en AgriParcel.

2.3 Qué ganas usando un modelo estándar

- **Interoperabilidad inmediata:** cualquier aplicación que entienda AgriParcel entiende tus parcelas, sin documentación adicional.
- **Diseño ya pensado:** los modelos recogen decisiones discutidas por la comunidad (unidades, relaciones, nombres).
- **Validación automática:** cada modelo incluye un JSON Schema contra el que comprobar los datos.
- **Semántica enlazada:** gracias a JSON-LD cada atributo tiene una URI única, lo que permite integrar datos de fuentes distintas y participar en espacios de datos.
- **Evolución controlada:** los modelos tienen versión y los cambios se revisan públicamente.

3 NGSI-LD en lo imprescindible

NGSI-LD representa el mundo como **entidades** (cosas) que tienen **propiedades** (valores), **relaciones** (enlaces a otras entidades) y **propiedades geográficas**. Todo ello en JSON-LD, es decir, JSON con significado semántico.

3.1 La entidad

Toda entidad tiene dos campos obligatorios:

- **id**: un identificador único con forma de URI. Por convención, `urn:ngsi-ld:<Tipo>:<identificador>`, por ejemplo `urn:ngsi-ld:AgriParcel:olivar-011`.
- **type**: el tipo de entidad, que coincide con el nombre del modelo: `AgriParcel`.

3.2 Los tres tipos de atributo

Property

Un valor: número, texto, booleano, lista u objeto. Puede llevar `unitCode` (unidad UN/CEFACT) y `observedAt` (momento de la medida).

Relationship

Un enlace al `id` de otra entidad. Se escribe con `object` en lugar de `value`. Es lo que convierte los datos en un grafo.

GeoProperty

Una geometría GeoJSON (Point, Polygon, MultiPolygon...). Permite consultas espaciales en el broker.

```
{
  "id": "urn:ngsi-ld:AgriParcel:olivar-011",
  "type": "AgriParcel",
  "slope": { "type": "Property", "value": 5.3, "unitCode": "P1" },
  "hasAgriCrop": { "type": "Relationship", "object": "urn:ngsi-ld:AgriCrop:olivo-hojiblanca" },
  "location": { "type": "GeoProperty", "value": { "type": "MultiPolygon", "coordinates": [...] } },
  "@context": [ "https://raw.githubusercontent.com/smart-data-models/dataModel.Agrifood/master/context.jsonld" ]
}
```

Una entidad mínima con un atributo de cada tipo, en forma normalizada.

3.3 El @context: de nombres cortos a significados únicos

En el JSON escribimos nombres cortos y legibles como `slope`. Pero "slope" podría significar la pendiente de un terreno, la de una recta o la de una rampa. El `@context` es un diccionario JSON-LD que asocia cada nombre corto con una URI completa y única:

`slope` > `@context` > <https://smartdatamodels.org/dataModel.Agrifood/slope>

Cuando el broker recibe una entidad, **expande** todos los nombres usando el contexto y los guarda por su URI. Al devolverla, los **comprime** de nuevo con el contexto que pida quien consulta. Así, dos sistemas pueden usar nombres cortos distintos (o idiomas distintos) y seguir refiriéndose al mismo concepto. Cada subject de Smart Data Models publica su propio `context.jsonld`.

Cuidado. Si un atributo no aparece en ningún contexto, el broker lo expande con un espacio de nombres por defecto. Funciona, pero pierde la semántica compartida. Por eso, cuando añadimos atributos nuevos, también hay que declarar sus términos (capítulo 7.9).

3.4 Normalizado frente a key-values

La misma entidad puede escribirse de dos maneras. La **forma normalizada** (la del ejemplo anterior) envuelve cada valor con su tipo de atributo y sus metadatos. La **forma key-values** (o *concise simplified*) es JSON plano, cómodo para aplicaciones pero sin metadatos:

```
{ "id": "urn:ngsi-ld:AgriParcel:olivar-011", "type": "AgriParcel",  
  "slope": 5.3, "hasAgriCrop": "urn:ngsi-ld:AgriCrop:olivo-hojiblanca", "location": { ... } }
```

Los **JSON Schema** de Smart Data Models describen la forma key-values; la forma normalizada se deriva de ella de manera mecánica. Por eso es importante que todo lo que el modelo necesita expresar quepa en key-values (lo veremos al hablar de la distancia a la estación, capítulo 7.6).

3.5 NGSI v2 y NGSI-LD

Existe una versión anterior de la API, **NGSI v2**, todavía muy usada (Orion). Es más sencilla, no usa `@context` y marca los tipos de otra forma (`"type": "Number"` , `"geo:json"`). Smart Data Models publica ejemplos en ambos formatos. Para trabajo nuevo y espacios de datos se recomienda NGSI-LD, que admiten brokers como **Orion-LD**, **Scorpio** y **Stellio**.

4 Anatomía de un modelo

Cada modelo es una carpeta en el repositorio de su subject. Solo tres o cuatro ficheros se escriben a mano; el resto los genera automáticamente el sistema de Smart Data Models a partir de ellos.

```
dataModel.Agrifood/
├─ context.jsonld          contexto de todo el subject (generado)
├─ agrifood-schema.json    definiciones comunes del subject (AgriFood-Commons)
├─ AgriParcel/
│   ├─ schema.json         ★ la definición formal del modelo
│   ├─ notes.yaml          ★ notas para la documentación
│   ├─ ADMIN.md            ★ datos administrativos de la contribución
│   ├─ examples/           ★ los cuatro ejemplos
│   │   ├─ example.json     NGSI v2 key-values
│   │   ├─ example-normalized.json  NGSI v2 normalizado
│   │   ├─ example.jsonld   NGSI-LD key-values
│   │   └─ example-normalized.jsonld NGSI-LD normalizado
│   ├─ model.yaml          versión YAML del schema (generado)
│   └─ doc/spec.md, spec_ES.md... especificación legible en varios idiomas (generado)
└─ LICENSE.md, swagger...  (generados)
```

★ = ficheros que escribe quien contribuye.

4.1 schema.json por dentro

Es un **JSON Schema** (versión 2020-12). Su estructura siempre sigue el mismo patrón: un bloque `allOf` que combina definiciones comunes con las propiedades propias del modelo.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$schemaVersion": "1.1.0",
  "title": "Smart Data Models - Agri Parcel",
  "allOf": [
    { "$ref": "../common-schema.json#/definitions/GSMA-Commons" },      id, name, description, dateCre
    { "$ref": "../common-schema.json#/definitions/Location-Commons" },  location, address, areaServed
    { "$ref": "../agrifood-schema.json#/definitions/AgriFood-Commons" }, relatedSource
    { "properties": {
      "area": { "type": "number", "minimum": 0,
        "description": "Property. Model:'http://schema.org/Number'. Units:'m2'. The area..." },
      ... } }
  ],
  "required": [ "id", "type", "location", "area", "hasAgriCrop" ]
}
```

Tres detalles merecen atención:

- **Los "Commons"** evitan repetir atributos genéricos en cada modelo. Por eso AgriParcel "tiene" `name`, `description` o `address` aunque no los declare.
- **La descripción tiene un formato fijo:** empieza por `Property.` o `Relationship.`, puede indicar `Model:'...'` (un tipo de schema.org) y `Units:'...'`, y termina con el texto explicativo. Las herramientas de SDM leen ese texto para generar la documentación y saber si un atributo es relación o propiedad.
- **required** es corto a propósito: solo lo imprescindible. Todo lo demás es opcional, lo que permite que el mismo modelo sirva para un huerto urbano o para un olivar de cien hectáreas.

4.2 Los demás ficheros escritos a mano

examples/

La misma entidad en los cuatro formatos. Deben validar contra el schema; son lo primero que mira quien quiere usar el modelo.

notes.yaml

Textos (`notesHeader` , `notesMiddle` , `notesFooter`) que se insertan en la especificación generada. Aquí se explican cambios de versión.

ADMIN.md

Quién contribuye, en qué contexto y qué se ha cambiado. Ayuda a los revisores.

5 Método para diseñar un modelo

Diseñar un Smart Data Model no empieza escribiendo un schema, sino entendiendo los datos y buscando lo que ya existe. Estos ocho pasos sirven para cualquier conjunto de datos.

Inventario › Buscar › Entidades › Mapeo › Tipos y unidades › Depurar › Validar › Contribuir

#	Paso	Pregunta que responde	Resultado
1	Inventario	¿Qué campos hay, de qué tipo, cuántos valores distintos, cuántos nulos?	Tabla de perfilado de datos
2	Buscar modelos existentes	¿Hay ya un modelo SDM que describa esta "cosa"? ¿Cubre la mayoría de campos?	Decisión: reutilizar, extender o crear
3	Identificar entidades	¿Qué valores se repiten y merecen existir por sí mismos? ¿Cómo se enlazan?	Diagrama de entidades y relaciones
4	Mapear atributos	¿A qué atributo estándar corresponde cada campo? ¿Cuáles faltan?	Tabla origen → destino
5	Tipos y unidades	¿Número, texto, lista, objeto? ¿En qué unidad? ¿Rango válido?	Definiciones del schema
6	Depurar	¿Qué campos son redundantes, derivables o de proceso interno?	Lista de descartes justificada
7	Validar	¿Todos los datos reales cumplen el schema? ¿Qué incidencias hay?	Entidades válidas e informe de calidad
8	Contribuir	¿Puede servir a otros? ¿Cómo se propone?	Pull request al subject

5.1 Reutilizar, extender o crear

El paso 2 es el más importante y el que más se salta. La regla práctica es:

Reutilizar tal cual

Si un modelo existente cubre todos tus campos. Lo ideal: cero cambios, interoperabilidad total.

Extender

Si un modelo cubre el concepto pero le faltan atributos. Se añaden como opcionales y se propone una nueva versión menor. **Es nuestro caso.**

Crear uno nuevo

Solo si ningún modelo describe esa "cosa". Hay que justificar por qué no encaja en los existentes.

Error frecuente. Crear un modelo "OlivarParcela" con todos los campos del fichero. Funcionaría en tu plataforma, pero ninguna herramienta de terceros sabría que es una parcela agrícola. Extender AgriParcel conserva todo lo que ya entienden los demás.

5.2 Criterios para separar entidades

Un valor merece convertirse en una entidad propia, enlazada mediante una relación, cuando se cumple alguna de estas condiciones:

- **Se repite** en muchos registros con idéntico significado (la variedad, la estación).

- **Tiene atributos propios** que no pertenecen al registro principal (una estación tiene coordenadas, variables medidas, un organismo gestor).
- **Tiene vida propia:** sus datos cambian independientemente, o generará otras entidades (la estación producirá observaciones meteorológicas diarias).
- **Otros sistemas querrán enlazarla** desde sus propios datos.

Es la misma lógica que la normalización de bases de datos relacionales: en lugar de repetir texto, se guarda una vez y se referencia. La diferencia es que en NGSI-LD la referencia es un `id` global, no una clave local.

6 El caso paso a paso

Aplicamos el método al fichero de parcelas: 104 features MultiPolygon en CRS84, 21 propiedades por feature.

6.1 Paso 1 · Inventario

Campo	Tipo	Distintos	Nulos	Observación
id	entero	104	0	1 a 104, identificador de origen
nombre	texto	1	0	vacío en todas
variedad	texto	5	0	Hojiblanca 50, Picual 31, Pajarero 19, Picudo 3, Nevadillo fino 1
marco	texto	6	1	"8x6" (61), "8x8" (23), "12x12" (13)...
fecha	texto	18	16	solo año; de 1985 a 2027
cubiera	texto	2	9	"sí" / "no"
pol_arregl	entero	1	0	siempre 1 (marca de procesado)
area_ha / area_m2	real	104	0	coherentes entre sí
pendiente_media_pct	real	78	0	derivado de MDT
orientacion_grados / _card	entero / texto	85 / 8	0	el cardinal se deduce de los grados
elev_media / min / max_m	real	~100	0	derivado de MDT
perfil_ns	lista	104	0	siempre 14 valores
estacion_siar / dist_siar_km	texto / real	1 / 95	0	siempre CO101
rc_catastro / sigpac_catastro	texto	80	0	listas separadas por ","
n_recintos_catastro	entero	3	0	= número de elementos de esas listas

Este perfilado ya revela casi todas las decisiones posteriores: campos vacíos o constantes que sobran, textos que en realidad son números o listas, valores repetidos que piden ser entidades y huecos que habrá que tratar como opcionales.

6.2 Paso 2 · Buscar modelos existentes

Una parcela agrícola con geometría y cultivo es exactamente lo que describe **AgriParcel**. Comparando campo a campo:

Lo que AgriParcel ya cubre

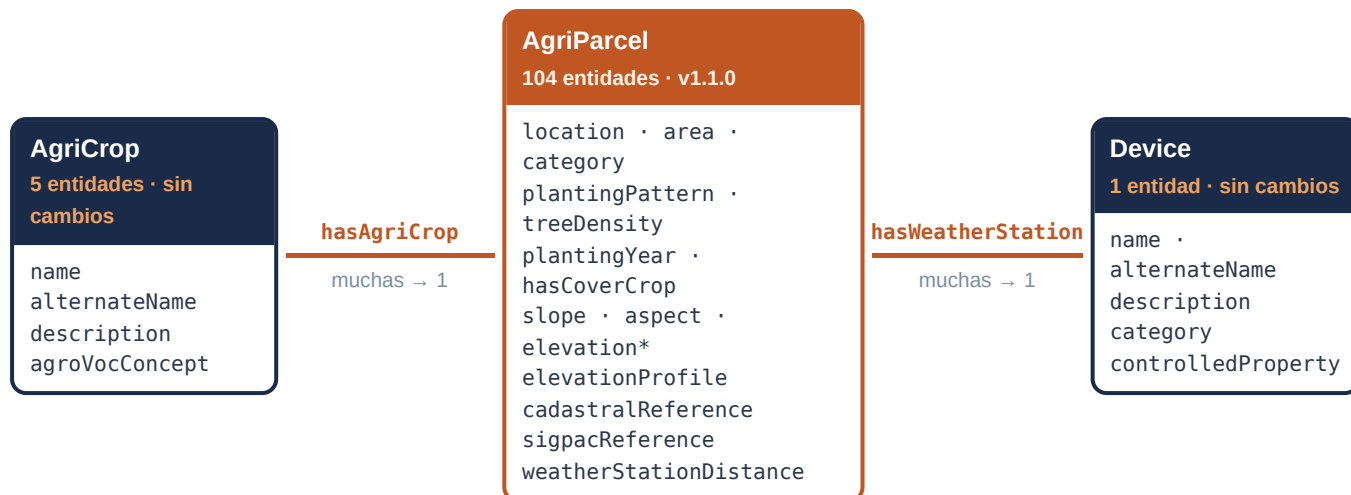
location (geometría), area, hasAgriCrop (cultivo), category, lastPlantedAt, hasAgriSoil, irrigationSystemType, belongsTo (explotación)...

Lo que no cubre

Marco de plantación y densidad, año de plantación, cubierta vegetal, pendiente, orientación, elevaciones, perfil, referencias catastrales y SIGPAC, estación agroclimática de referencia.

Conclusión: **extender AgriParcel** con atributos opcionales. Para la variedad y la estación ya existen **AgriCrop** y **Device**, que se reutilizan sin cambios.

6.3 Paso 3 · Entidades y relaciones



Modelo de entidades resultante: 110 entidades en total. Las flechas son relaciones NGSI-LD.

La variedad cumple los criterios del capítulo 5.2: se repite (50 veces Hojiblanca), tiene atributos propios (especie, concepto AGROVOC, características varietales que se podrían añadir) y otros sistemas querrán enlazarla (un modelo de rendimiento por variedad, por ejemplo). Lo mismo la estación SIAR, que además será el origen de futuras entidades **weatherObserved** con los datos climáticos diarios.

6.4 Pasos 4 a 6 · Mapeo completo

Campo de origen	Atributo destino	Tipo	unitCode	Transformación
geometry	location	GeoProperty	—	se copia tal cual (ya está en WGS84)
id	id	URI	—	urn:ngsi-ld:AgriParcel:olivar-011
—	category	texto	—	constante orchard (cultivo leñoso)
area_m2	area	número	MTK (m²)	redondeo a 2 decimales
variedad	hasAgriCrop	Relationship	—	"HOJIBLANCA" → urn:ngsi-ld:AgriCrop:olivo-hojiblanca
marco	plantingPattern	objeto	MTR	"8x6" → {rowSpacing: 8, treeSpacing: 6}
marco	treeDensity	número	árboles/ha	10000 / (8 × 6) = 208,3
fecha	plantingYear	entero	—	"2012" → 2012
cubiera	hasCoverCrop	booleano	—	"si" → true, "no" → false
pendiente_media_pct	slope	número	P1 (%)	directo
orientacion_grados	aspect	número	DD (grados)	directo
elev_media/min/max_m	elevationMean / Min / Max	número	MTR	directo
perfil_ns	elevationProfile	lista de números	MTR	directo
estacion_siar	hasWeatherStation	Relationship	—	"CO101 - ..." → urn:ngsi-ld:Device:SIAR-CO101
dist_siar_km	weatherStationDistance	número	KMT	directo
rc_catastro	cadastralReference	lista de texto	—	separar por ";"
sigpac_catastro	sigpacReference	lista de texto	—	separar por ";"
nombre	descartado			vacío en todas las parcelas
pol_arregl	descartado			marca interna de procesado, constante
area_ha	descartado			redundante con area
orientacion_card	descartado			se calcula a partir de aspect
n_recintos_catastro	descartado			es la longitud de las listas

Los códigos de unidad

NGSI-LD expresa las unidades con los códigos comunes de la recomendación 20 de UN/CEFACT, de tres caracteres, en el campo `unitCode`. Los usados aquí son: `MTK` metro cuadrado, `MTR` metro, `KMT` kilómetro,

P1 porcentaje, DD grado sexagesimal y HAR hectárea. En la forma key-values el unitCode desaparece; por eso la unidad también se declara en la descripción del schema (`Units: 'm2'`).

7 Las decisiones de diseño, explicadas

Un modelo de datos es un conjunto de decisiones. Documentar el porqué de cada una es tan importante como el propio schema: es lo que permite a otros entenderlo, aceptarlo en una revisión o cambiarlo con criterio.

7.1 La superficie va en metros cuadrados

El fichero trae la superficie en hectáreas y en metros cuadrados. Lo intuitivo en agricultura sería usar hectáreas, pero el schema oficial de AgriParcel define `area` con unidad `m2`. **Cuando se extiende un modelo, se respetan sus definiciones:** si usáramos hectáreas, cualquier aplicación que sume superficies de parcelas de distintas fuentes obtendría resultados 10.000 veces equivocados. Las hectáreas se pueden mostrar en la interfaz, pero el dato se almacena como dice el estándar.

7.2 El marco de plantación como objeto

Guardar `"8x6"` como texto obligaría a cada consumidor a trocearlo. Un objeto con dos números, `{rowSpacing: 8, treeSpacing: 6}`, permite filtrar ("marcos con calle mayor de 7 m") y calcular. Añadimos además `treeDensity`, derivado, porque la densidad (árboles/ha) es la magnitud que se usa en agronomía para distinguir olivar tradicional, intensivo y superintensivo. Aquí se hace una excepción consciente a la regla de "no guardar derivados", porque el dato tiene significado agronómico propio y puede conocerse aunque no se conozca el marco.

Supuesto por confirmar. Se ha interpretado que el primer número es la distancia entre filas y el segundo la distancia entre árboles de la misma fila. En marcos cuadrados da igual, pero en "8x6" no.

7.3 Año de plantación y no fecha

AgriParcel ya tiene `lastPlantedAt`, pero es de tipo fecha-hora. Convertir "2012" en `2012-01-01T00:00:00Z` inventaría una precisión que no existe: parecería que se plantó el 1 de enero. Es preferible un atributo `plantYear` entero que diga exactamente lo que se sabe. Si en el futuro se conoce la fecha completa, se usa `lastPlantedAt`.

7.4 Cubierta vegetal como booleano

"sí" y "no" son textos; `true / false` son un tipo de dato que cualquier sistema entiende sin ambigüedad (¿"Sí", "Sí", "sí", "yes"?). Los valores nulos no se convierten en `false`: simplemente **se omite el atributo**. En NGSI-LD no se escriben atributos vacíos; la ausencia significa "no se sabe", que es distinto de "no".

7.5 Variedades como AgriCrop enlazadas a AGROVOC

Cada variedad se convierte en una entidad `AgriCrop` con un identificador legible (`urn:ngsi-ld:AgriCrop:olivo-picual`) y con `agroVocConcept` apuntando al concepto de AGROVOC para *Olea europaea* (http://aims.fao.org/aos/agrovoc/c_5339). AGROVOC es el tesoro multilingüe de la FAO: enlazarlo significa que cualquier sistema, en cualquier idioma, sabe que se trata del olivo. Las características de cada variedad (vigor, productividad, época de maduración) se pueden añadir después en un único sitio.

7.6 La estación SIAR como Device, y la distancia como atributo plano

La estación se modela con el modelo genérico `Device` de `dataModel.Device`, indicando en `controlledProperty` las variables que mide (temperatura, humedad, precipitación, radiación, viento). La parcela la referencia con `hasWeatherStation`, siguiendo la convención `has...` que usa todo Agrifood (`hasAgriCrop`, `hasAgriSoil`, `hasDevice`).

NGSI-LD permite poner una propiedad *dentro* de una relación, y la distancia a la estación es conceptualmente eso: una característica del enlace. Sin embargo, los JSON Schema de SDM describen la forma key-values, donde no existen subpropiedades. Si la distancia fuera solo una subpropiedad, se perdería al pasar a key-values y no se podría validar. Por eso se declara como atributo plano, `weatherStationDistance`. Es un buen ejemplo de cómo las herramientas del ecosistema condicionan el diseño.

¿Por qué no `hasDevice`? AgriParcel ya tiene `hasDevice`, pensado para sensores o actuadores instalados en la parcela. Una estación a 9 km no es un dispositivo de la parcela, sino una fuente de referencia; mezclarlos confundiría a quien consulte los datos.

7.7 Referencias catastrales y SIGPAC como listas

Una parcela agronómica puede solapar varias parcelas catastrales. El texto con punto y coma se convierte en una lista real, y el campo que contaba elementos desaparece porque la propia lista ya lo dice. Así es imposible que "el contador diga 2 y haya 3". El schema incluye un patrón que comprueba el formato básico (14 o 20 caracteres alfanuméricos para Catastro; segmentos separados por dos puntos para SIGPAC).

7.8 Sin dirección postal, de momento

Sería natural añadir `address` con el municipio a partir del código SIGPAC. Pero en los datos aparecen dos códigos municipales (013 y 042) y los códigos de municipio de SIGPAC y Catastro no siempre coinciden con los del INE. Antes que arriesgar un municipio incorrecto, se omite: **es mejor un dato ausente que un dato inventado**. Con una tabla de correspondencia fiable se puede añadir en el script.

7.9 Los términos nuevos en el `@context`

Los 16 términos nuevos (`slope`, `plantingPattern`, `rowSpacing`...) no están aún en el `context.jsonld` oficial de Agrifood. Mientras no se acepte la contribución, se declaran **en línea** dentro del propio `@context` de cada entidad, como un segundo elemento del array:

```
"@context": [
  "https://raw.githubusercontent.com/smart-data-models/dataModel.Agrifood/master/context.jsonld",
  {
    "slope": "https://smartdatamodels.org/dataModel.Agrifood/slope",
    "plantingPattern": "https://smartdatamodels.org/dataModel.Agrifood/plantingPattern", ... }
]
```

Se usan ya las URI que tendrán cuando se integren en el subject, de modo que cuando se publique el contexto oficial bastará con quitar el objeto en línea: los datos almacenados en el broker no cambian.

8 La misma parcela en cuatro formatos

Estos son los ejemplos del paquete (AgriParcel/examples/), generados a partir de la parcela 11. La geometría se abrevia.

8.1 NGSI-LD key-values · example.jsonld

```
{
  "id": "urn:ngsi-ld:AgriParcel:olivar-011", "type": "AgriParcel",
  "location": { "type": "MultiPolygon", "coordinates": [[[-4.543056, 37.513413], ...]] },
  "category": "orchard",
  "area": 97986.08,
  "hasAgriCrop": "urn:ngsi-ld:AgriCrop:olivo-hojiblanca",
  "plantingPattern": { "rowSpacing": 8.0, "treeSpacing": 8.0 },
  "treeDensity": 156.2,
  "plantingYear": 2012,
  "hasCoverCrop": true,
  "slope": 5.3, "aspect": 287,
  "elevationMean": 419.5, "elevationMin": 405.0, "elevationMax": 429.8,
  "elevationProfile": [424.4, 422.9, 419.2, 418.3, 418.0, 417.7, 417.4, 417.6, 419.1, ...],
  "hasWeatherStation": "urn:ngsi-ld:Device:SIAR-C0101",
  "weatherStationDistance": 9.74,
  "cadastralReference": ["14013A001001530000GH", "14013A001000510000GJ"],
  "sigpacReference": ["14:013:0:0:1:153", "14:013:0:0:1:51"],
  "@context": [ ".../dataModel.Agrifood/master/context.jsonld", { 16 términos nuevos } ]
}
```

8.2 NGSI-LD normalizado · example-normalized.jsonld (extracto)

```
{
  "id": "urn:ngsi-ld:AgriParcel:olivar-011", "type": "AgriParcel",
  "location": { "type": "GeoProperty", "value": { "type": "MultiPolygon", "coordinates": [...] } },
  "area": { "type": "Property", "value": 97986.08, "unitCode": "MTK" },
  "hasAgriCrop": { "type": "Relationship", "object": "urn:ngsi-ld:AgriCrop:olivo-hojiblanca" },
  "plantingPattern": { "type": "Property",
    "value": { "rowSpacing": 8.0, "treeSpacing": 8.0 }, "unitCode": "MTR" },
  "hasCoverCrop": { "type": "Property", "value": true },
  "slope": { "type": "Property", "value": 5.3, "unitCode": "P1" },
  "aspect": { "type": "Property", "value": 287, "unitCode": "DD" },
  "hasWeatherStation": { "type": "Relationship", "object": "urn:ngsi-ld:Device:SIAR-C0101" },
  "weatherStationDistance": { "type": "Property", "value": 9.74, "unitCode": "KMT" },
  ...
}
```

8.3 NGSI v2 · example.json y example-normalized.json

La versión v2 key-values es idéntica a la LD key-values pero sin `@context`. La normalizada v2 usa tipos distintos:

```

"area": { "type": "Number", "value": 97986.08 },
"hasCoverCrop": { "type": "Boolean", "value": true },
"hasAgriCrop": { "type": "Relationship", "value": "urn:ngsi-ld:AgriCrop:olivo-hojiblanca" },
"location": { "type": "geo:json", "value": { "type": "MultiPolygon", ... } },
"plantingPattern": { "type": "StructuredValue", "value": { "rowSpacing": 8.0, "treeSpacing": 8.0 } }

```

8.4 Las otras dos entidades

```

{ "id": "urn:ngsi-ld:AgriCrop:olivo-hojiblanca",
  "type": "AgriCrop",
  "name": "Olivo Hojiblanca",
  "alternateName": "HOJIBLANCA",
  "description": "Olivo (Olea europaea L.),
                 variedad Hojiblanca",
  "agroVocConcept":
    "http://aims.fao.org/aos/agrovoc/c_5339" }

```

```

{ "id": "urn:ngsi-ld:Device:SIAR-C0101",
  "type": "Device",
  "name": "IFAPA Centro de Cabra",
  "alternateName": "C0101",
  "category": ["sensor"],
  "controlledProperty": ["temperature",
                          "humidity", "precipitation",
                          "solarRadiation", "windSpeed",
                          "windDirection"] }

```

9 Validación y calidad de datos

El JSON Schema es un **contrato**: si una entidad lo cumple, cualquier consumidor puede confiar en su estructura. Validar no es un trámite final; es la forma de descubrir problemas en los datos que a simple vista pasan desapercibidos.

9.1 Cómo se valida

El script `validate.py` comprueba cada entidad key-values contra su schema: AgriParcel contra el schema 1.1.0 del paquete y AgriCrop y Device contra los oficiales. Los `$ref` a los Commons se resuelven descargándolos del repositorio de Smart Data Models, igual que haría cualquier validador externo. Resultado:

104 AgriParcel ✓ · 5 AgriCrop ✓ · 1 Device ✓ · 110 entidades válidas

9.2 Lo que encontró el proceso

El conversor genera además un informe de calidad (`informe_calidad.md`) con 30 incidencias. Ninguna impide cargar los datos, pero todas merecen una revisión:

Incidencia	Parcelas	Por qué importa
Referencia catastral mal formada	13 y 49	Terminan en "G0". Los dos caracteres de control del Catastro son siempre letras; probablemente es una letra O escrita como cero. Con esa referencia no se localizaría la parcela en la Sede del Catastro. Se detectó porque el primer patrón del schema era estricto.
Año de plantación futuro (2027)	35 y 85	¿Plantaciones previstas o error de tecleo? Si son previstas, conviene expresarlo explícitamente, por ejemplo con el estado del cultivo.
Sin año de plantación	16 parcelas	Limita análisis por edad del olivar.
Sin dato de cubierta	1 a 9	Todas consecutivas: sugiere un lote cargado antes de que existiera el campo.
Sin marco de plantación	82	Sin marco no hay densidad.

Una lección sobre los patrones. Se decidió relajar el patrón del schema (`[0-9A-Z]{6}` en los caracteres finales) para que el modelo no rechace datos reales imperfectos, y trasladar la comprobación estricta al informe de calidad. Un modelo estándar debe ser tolerante; la limpieza de datos es responsabilidad de cada fuente.

9.3 Por qué los nulos se omiten

Cuando falta un dato, el conversor no escribe el atributo. Esto tiene tres ventajas: el schema no necesita admitir `null` en cada campo, el broker no almacena atributos vacíos que luego aparecerían en consultas, y se distingue claramente "no se sabe" de un valor real. Solo los atributos de `required` son imprescindibles; en AgriParcel, `location`, `area` y `hasAgriCrop`, y las 104 parcelas los tienen.

10 El paquete generado y cómo usarlo

El fichero `agriparcel-olivar-sdm.zip` contiene el modelo, los scripts y los datos ya convertidos.

```
agriparcel-olivar-sdm/
├── README.md                      resumen y decisiones pendientes
├── context-extension.jsonld       los 16 términos nuevos
├── AgriParcel/
│   ├── schema.json              AgriParcel 1.1.0 (oficial + 14 atributos)
│   ├── notes.yaml · ADMIN.md · model.yaml
│   └── examples/                los 4 formatos del capítulo 8
├── conversion/
│   ├── geojson_to_nginxld.py    conversor (solo biblioteca estándar)
│   └── validate.py              validador (jsonschema + referencing)
└── output/
    ├── entities_nginxld_normalized.jsonld  110 entidades para el broker
    ├── entities_nginxld_keyvalues.jsonld
    ├── entities_keyvalues.json           para validar
    └── informe_calidad.md
```

10.1 Convertir y validar

```
# conversión (genera los ficheros de output/ y el informe)
python geojson_to_nginxld.py parcelas_agronicas.geojson --out ../output

# validación contra los schemas
pip install jsonschema referencing
python validate.py ../output/entities_keyvalues.json ../AgriParcel/schema.json
```

Opciones útiles del conversor: `--id-prefix` para cambiar el prefijo de los identificadores y `--current-year` para ajustar la detección de años futuros. Si el GeoJSON viniera en otra proyección (por ejemplo ETRS89 UTM 30N), el script se detiene y pide reproyectarlo, porque NGSI-LD exige coordenadas WGS84.

10.2 Cargar en el context broker

```
python geojson_to_nginxld.py parcelas_agronicas.geojson --out ../output \
    --broker http://localhost:1026 --tenant dsagro
```

El script envía las entidades normalizadas en lotes de 50 a `POST /ngsi-ld/v1/entityOperations/upsert` con `Content-Type: application/ld+json`. *Upsert* significa que crea las que no existen y actualiza las que sí, así que se puede ejecutar varias veces sin duplicar. La cabecera `NGSILD-Tenant` separa los datos por inquilino si el broker lo admite.

10.3 Consultas de ejemplo

Una vez cargados, el modelo permite preguntas que con el GeoJSON exigían programar:

```
# parcelas con pendiente superior al 15 %
GET /ngsi-ld/v1/entities?type=AgriParcel&q=slope>15

# parcelas de Picual
GET /ngsi-ld/v1/entities?type=AgriParcel&q=hasAgriCrop=="urn:ngsi-ld:AgriCrop:olivo-picual"

# parcelas a menos de 2 km de un punto
GET /ngsi-ld/v1/entities?type=AgriParcel&georel=near;maxDistance==2000
    &geometry=Point&coordinates=[-4.53,37.52]

# solo algunos atributos, en formato key-values
GET /ngsi-ld/v1/entities?type=AgriParcel&attrs=area,slope,plantingYear&options=keyValues
```

En las consultas se usan los nombres cortos; para que el broker los entienda, la petición debe indicar el mismo contexto con una cabecera `Link` que apunte al context.jsonld (o, con el contexto en línea, usar las URI completas hasta que el contexto oficial esté publicado).

11 Contribuir a Smart Data Models

Los modelos evolucionan gracias a contribuciones de quienes los usan. Proponer la extensión de AgriParcel convierte un trabajo local en un estándar que otros proyectos de olivar, viñedo o cítricos podrán aprovechar.

11.1 Versionado

Los modelos siguen versionado semántico `MAYOR.MENOR.PARCHE` :

Parche (1.0.2 → 1.0.3)

Correcciones de descripciones o erratas, sin cambios en la estructura.

Menor (1.0.2 → 1.1.0)

Se añaden atributos opcionales. Los datos antiguos siguen siendo válidos. **Nuestra propuesta.**

Mayor (1.x → 2.0.0)

Cambios que rompen: renombrar, cambiar tipos, hacer obligatorio algo. Se evitan siempre que sea posible.

Como los 14 atributos nuevos son opcionales, cualquier AgriParcel que hoy exista en cualquier broker del mundo sigue siendo válido con la versión 1.1.0. Esta **retrocompatibilidad** es el argumento principal para que la propuesta se acepte.

11.2 El proceso

Fork de dataModel.Agrifood › Sustituir schema, notes, ejemplos › Pull request › Revisión › Merge › Regeneración automática

1. Hacer un *fork* del repositorio `smart-data-models/dataModel.Agrifood` en GitHub.
2. Sustituir en la carpeta `AgriParcel/` los ficheros `schema.json`, `notes.yaml` y `examples/*` por los del paquete.
3. Abrir un *pull request* explicando el caso de uso (el texto de `ADMIN.md` sirve de base): qué atributos se añaden, por qué son útiles más allá de este proyecto y que el cambio es retrocompatible.
4. Atender los comentarios de los revisores. Es habitual que sugieran otros nombres o que un atributo encaje mejor en otro modelo.
5. Tras el *merge*, el sistema regenera `model.yaml`, la especificación en varios idiomas y el `context.jsonld` del subject. A partir de ese momento el contexto en línea de nuestras entidades se puede retirar.

11.3 Argumentos para la revisión

- **Generalidad:** marco, densidad, cubierta y año de plantación sirven para cualquier cultivo leñoso (olivar, viñedo, frutales, cítricos), no solo para este caso.
- **Topografía:** pendiente, orientación y elevación son determinantes en agronomía (erosión, insolación, heladas) y se obtienen de modelos digitales de elevación públicos.
- **Referencias administrativas:** Catastro y SIGPAC son la base de la PAC en España; atributos equivalentes podrían generalizarse a otros países.
- **Coste nulo para los usuarios actuales:** todo es opcional.

11.4 Uso local mientras tanto

No hace falta esperar a la aceptación: el schema del paquete se puede registrar en una plataforma propia (por ejemplo, el portal del Aula con su formato de etiquetas por atributo) y los datos pueden cargarse ya en el broker con el contexto en línea. Si la revisión cambia algún nombre, el conversor se ajusta en minutos y se vuelve a ejecutar el upsert.

12 Glosario y lista de comprobación

AGROVOC

Tesaurus multilingüe de la FAO sobre agricultura y alimentación; cada concepto tiene una URI estable.

Commons

Conjuntos de atributos genéricos (GSMA-Commons, Location-Commons...) que los modelos incorporan con

`$ref`.

Context broker

Servidor que almacena entidades y ofrece la API NGSI-LD (Orion-LD, Scorpio, Stellio).

@context

Diccionario JSON-LD que traduce nombres cortos a URI únicas.

Entidad

Representación digital de una cosa, con `id` y `type`.

GeoProperty

Atributo cuyo valor es una geometría GeoJSON.

JSON Schema

Estándar para describir y validar la estructura de documentos JSON.

JSON-LD

JSON con semántica enlazada mediante `@context`.

Key-values

Forma simplificada de una entidad: atributo → valor, sin metadatos.

NGSI-LD

API y modelo de información de contexto normalizados por ETSI.

Normalizado

Forma completa: cada atributo con su tipo (Property, Relationship, GeoProperty) y metadatos.

Property / Relationship

Atributo con valor / atributo que enlaza con otra entidad.

Subject

Repositorio de Smart Data Models que agrupa modelos relacionados (`dataModel.Agrifood`).

unitCode

Código UN/CEFACT de tres caracteres que indica la unidad de un valor.

Upsert

Operación que crea o actualiza según exista o no la entidad.

URN

Identificador con forma `urn:ngsi-ld:Tipo:id`.

Lista de comprobación para tu próximo modelo

- ☐ He perfilado los datos: tipos, valores distintos y nulos.
- ☐ He buscado en Smart Data Models un modelo que ya describa esa cosa.
- ☐ He decidido reutilizar, extender o crear, y sé justificarlo.
- ☐ He separado en entidades los valores que se repiten o tienen vida propia.
- ☐ Cada atributo tiene tipo, unidad y descripción en el formato SDM.
- ☐ Respeto las unidades y definiciones del modelo que extiendo.
- ☐ He descartado campos vacíos, constantes, redundantes o de proceso.
- ☐ No invento precisión (fechas, municipios) que los datos no tienen.
- ☐ Los nulos se omiten, no se escriben.
- ☐ Todo lo necesario cabe en key-values.
- ☐ Los términos nuevos están en un `@context`.
- ☐ Todas las entidades validan y tengo un informe de calidad.

En resumen · Un Smart Data Model es un **vocabulario compartido** (nombres, tipos, unidades y relaciones) con un **contrato verificable** (el JSON Schema) y un **significado global** (el @context). Para las parcelas de olivar no hizo falta inventar nada: se **extendió AgriParcel** con 14 atributos opcionales, se enlazaron variedades y estación como entidades propias, y el resultado son **110 entidades válidas** listas para el broker y una propuesta retrocompatible para la comunidad.



UNIVERSIDAD
DE
CÓRDOBA

Aula de Transformación Digital FIWARE · Universidad de Córdoba

Referencias: Smart Data Models (smartdatamodels.org · github.com/smart-data-models), especificación ETSI NGSI-LD, tesauro AGROVOC (FAO), recomendación 20 de UN/CEFACT.